Agentic AI Hackathon · IIT Bhubaneswar

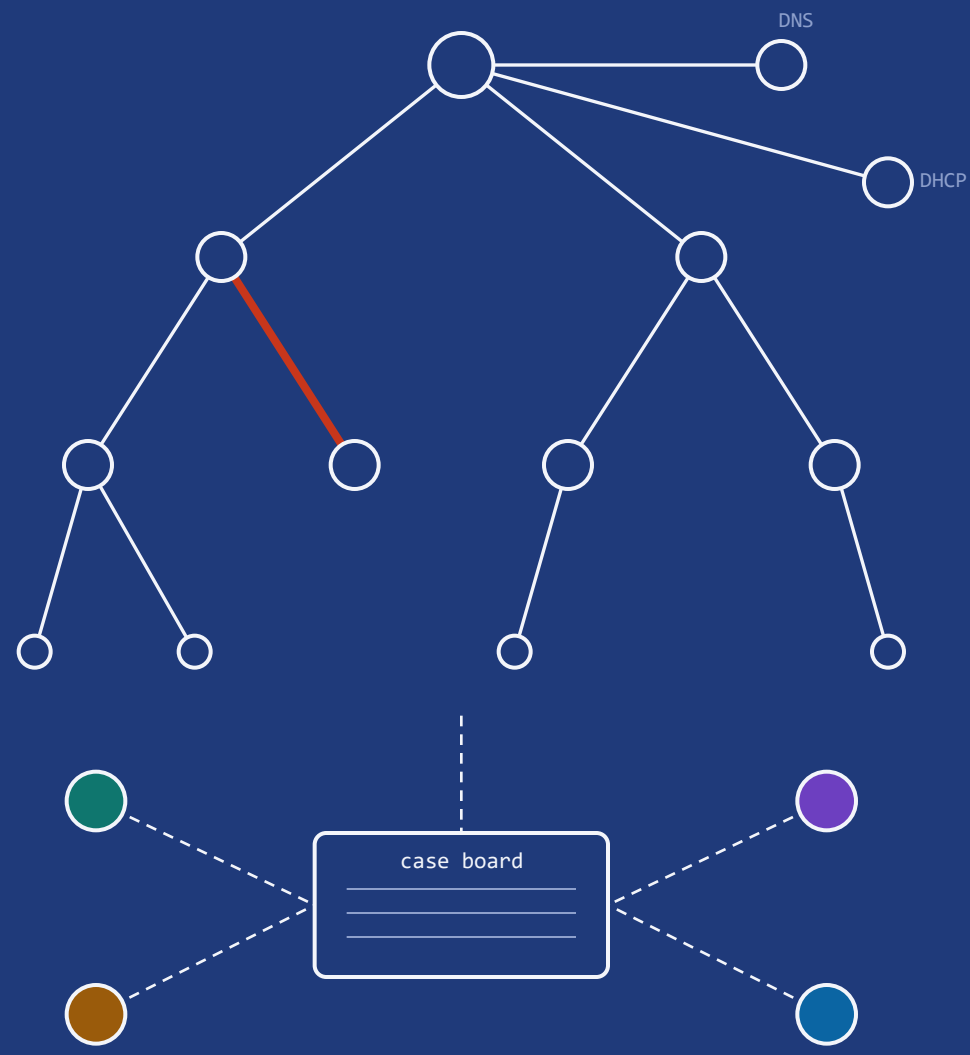
NetCoop AI

Network faults diagnosed by cooperating agents, and *every claim cites its proof*.

Specialist agents analyse a campus network's telemetry, exchange typed messages on a shared case board, and publish one explainable report, or honestly say there isn't enough evidence.

§1 Problem · §2 How it works · §3 Agents in action
§4 The report · §5 Robustness · §6 Results

Team
Havoc



the fault (red link) and the four specialist agents reporting to one case board

§1 Problem -> §2 How it works

Specialist agents reach one verdict *by citing each other's evidence*

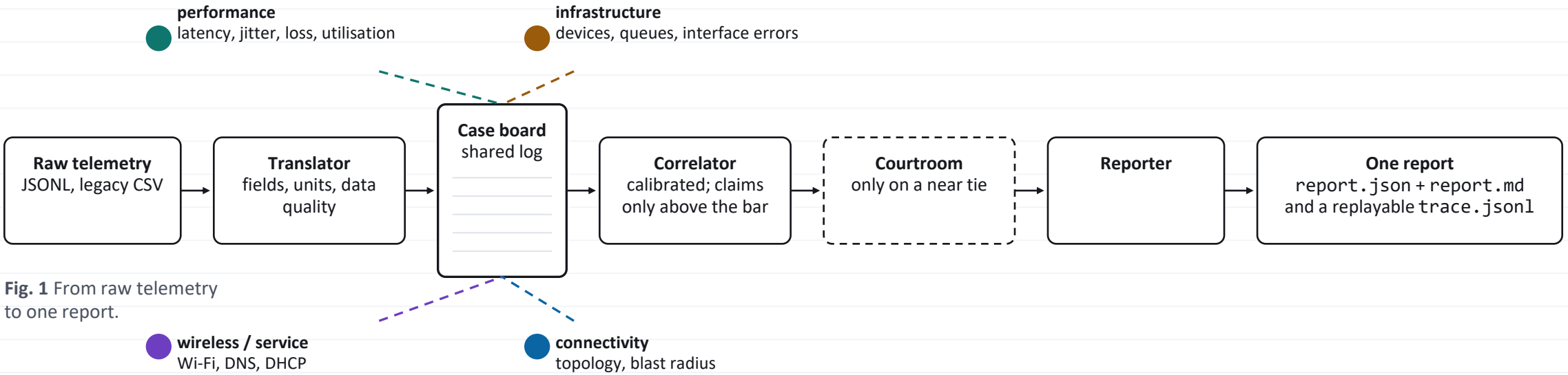


Fig. 1 From raw telemetry to one report.

Problem. Evidence of a network fault is scattered across devices, links, Wi-Fi and services, in different formats.

Approach. Agents never call each other: they only post typed messages (13 types, such as `CLUE`, `HYPOTHESIS`, `REQUEST`, `RESPONSE`, `SUPPORT`, `CONTRADICT`, `VERDICT`) on a shared board.

Guarantee. The same input and configuration give a byte-identical trace[1].

[1] `netcoop replay` verifies every trace (tests/test_orchestrator.py)

§3 Agents in action

One diagnosis, *every message on the record*

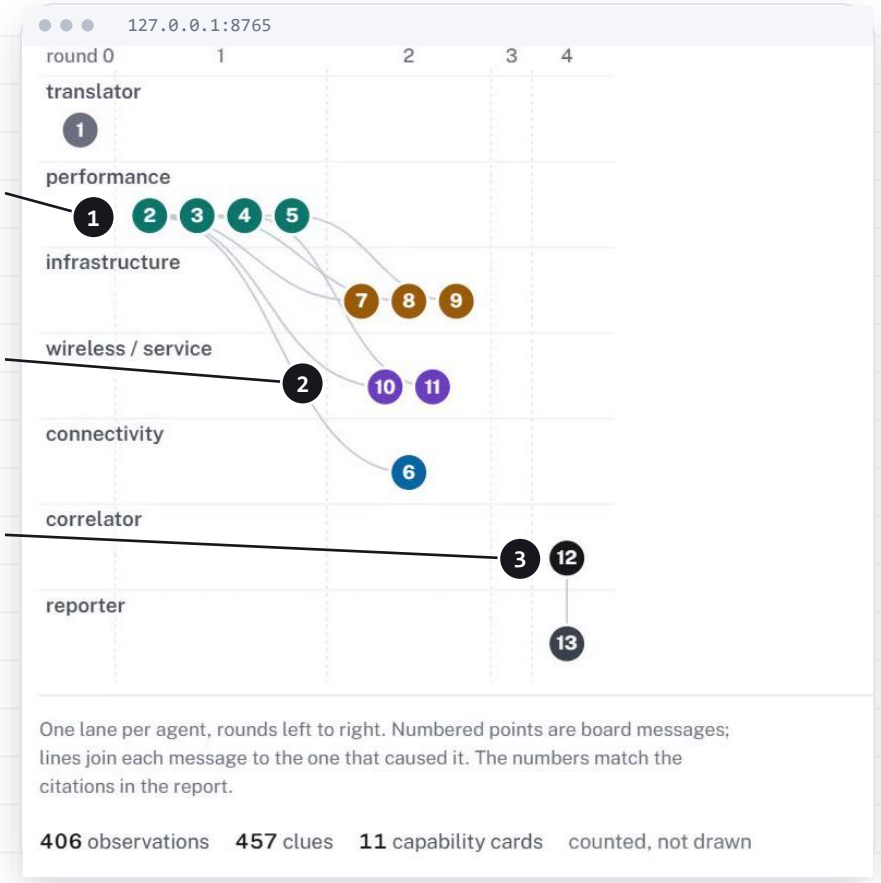


Fig. 2 The agents' message exchange for a link-congestion run: one lane per agent, rounds left to right, lines from each message to its cause.

1. The Translator reads raw telemetry into **406 observations**[1].
2. Specialists compare each metric with calibrated baselines and post **457 clues**[1], including the "checked, normal" ones.
3. Performance posts a HYPOTHESIS (link congestion), then **3** REQUESTs for evidence it cannot see itself.
4. Infrastructure and wireless RESPOND; three agents post SUPPORT stances. Connectivity confirms the symptoms sit inside the link's blast radius.
5. The correlator issues the VERDICT: diagnosed, confidence 1.00.

887 messages, 3 collaboration rounds[1]

[1] `netcoop run data/samples/link_congestion__s500__i1.0__n0.0__m0.0`

§4 The final report

The report reads like a paper: *each claim links to its proof*

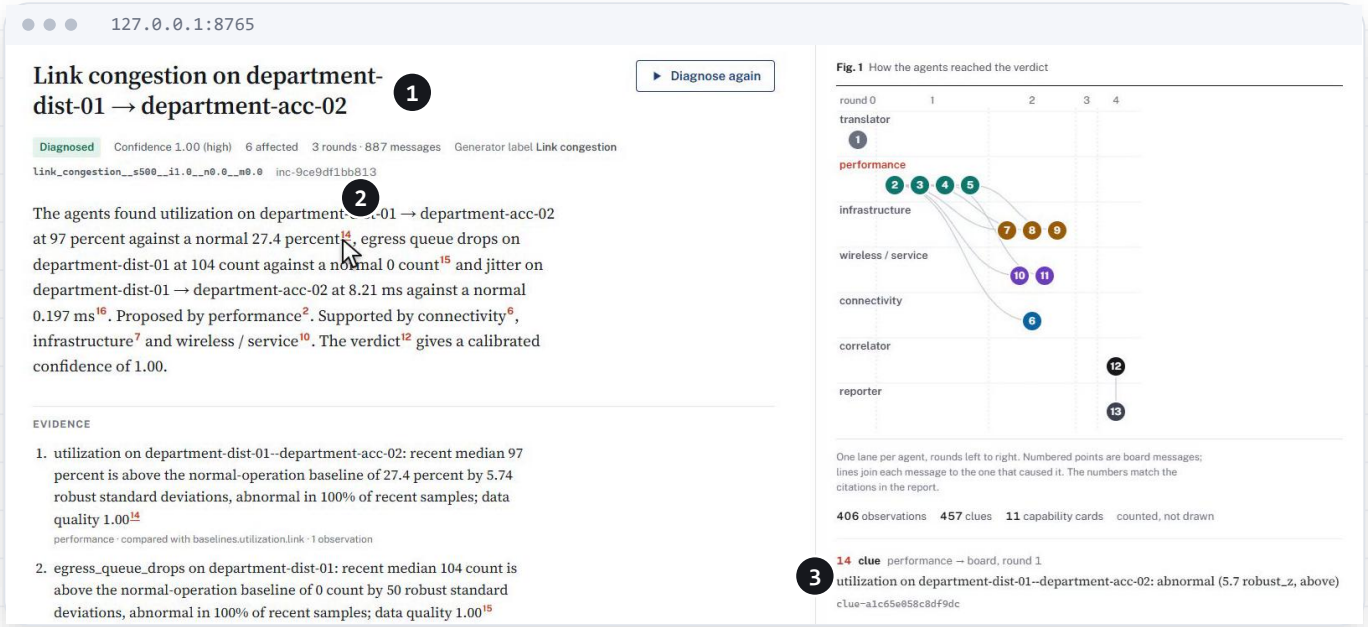


Fig. 3 Hovering a red citation lights up the message that proves it.

1 root cause and calibrated confidence

2 a red citation

3 the cited message, highlighted

- **Root cause at a specific link:** "Link congestion on department-dist-01 → department-acc-02"[1]
- **Evidence with its baseline:** utilisation 97% against a normal 27.4%; jitter 8.21 ms against 0.197 ms; 104 egress queue drops against 0[1]
- **Recommended actions,** for human review only; nothing is executed automatically
- **Also:** affected devices, alternatives considered, data quality, limitations



Not enough evidence

A faint fault: NetCoop abstains instead of guessing.



Courtroom: deferred

A fault type never seen in training: the judge defers rather than guess.

[1] report.md of the link-congestion sample (deliverables/4_sample_data_and_test_cases/example_reports)

§5 Interoperability and robustness

Messy, foreign data *bends but never breaks* the pipeline

```
PS C:\netcoop-ai> netcoop translate
data/samples/interop_failure__s500__i0.5__n0.1__m0.0
dialects: {'device_stats.csv': 'legacy_v1', 'link_stats.csv': 'legacy_v1',  (1)
'service_checks.csv': 'legacy_v1', 'topology_edges.csv': 'legacy_v1',
'wifi_stats.csv': 'legacy_v1'}
stats: {'files': 5, 'tables': 5, 'topology_tables': 1, 'series': 466,
'samples': 13980, 'errors': 0}
```

Fig. 4 A legacy CSV export: 5 files, 466 series, 13,980 samples, 0 errors[3].

(1) dialect detected: legacy_v1

Table 1 Precision of claims as stacked format mutations increase.

mutations stacked	runs	precision of claims
0	48	97.5%
1	48	97.1%
2	50	96.3%
3	**46**	**95.7%**[1]

Bad input → typed ERROR messages and lower data-quality scores; the Translator never raises on input (22 kinds of corruption tested[2]).

Agent failure or timeout → that agent is isolated and its missing evidence is listed in the report.

Fatal problem → never a false "no fault".

[1] docs/EVALUATION.md, robustness · [2] docs/INTEROP_RESULTS.md · [3] `netcoop translate`, interop sample

§6 Results and what we deliver

On 1,080 fresh runs, *98.6% of claims are right*

98.6%[1]	77.2%[1]	100%[1]	0[2]
precision of claims (12 wrong of 846)	accuracy	root localisation	wrong claims on 120 runs of never-seen fault types

Table 2 Full system, fresh test suite; last row: without → with courtroom.

metric	value
abstention rate	21.7%
macro-F1	0.860
courtroom invoked	227 runs
wrong claims on ambiguous cases	**32 → 0**[3]

Deliverables

1. Working prototype: the `netcoop` CLI and a local dashboard (`netcoop serve`)
2. Source, setup, dependencies, and **246 tests** passing in CI[4]
3. Architecture documentation: agents, protocol, orchestration, correlation, errors, workflow
4. Nine labelled samples: normal operation plus eight fault scenarios
5. A 4 min 25 s demo video
6. Extensible: new agents and faults are data, external agents join over MCP, runs pause and resume, humans review actions

NetCoop AI: *every claim cites its proof.*

github.com/suvronilchatterjee7-create/netcoop-ai

[1] docs/EVALUATION.md · [2] ambiguity suite, seeds 100–109 · [3] docs/COURTROOM_RESULTS.md · [4] pytest, 246 passed